Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Probabilité conditionnelle :

La probabilité avancée est essentielle pour la compréhension approfondie des données et la prise de décision dans divers domaines. Nous explorerons également la manière dont ces concepts peuvent être appliqués en pratique en utilisant Python

Définition et formule de La probabilité conditionnelle :

• La probabilité conditionnelle est la probabilité qu'un événement A se produise, sachant que l'événement B s'est déjà produit.

Elle est représentée par P(A|B) et peut être interprétée comme la probabilité de A sous la condition que B se soit produit.

• La formule mathématique de la probabilité conditionnelle est donnée par:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Exemples et cas d'utilisation dans l'analyse de données :

• Probabilité de pluie sachant que le ciel est couvert .

• Probabilité de défaut d'un produit sachant qu'il a été fabriqué sur une machine spécifique .

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Théorème de Bayes

Le théorème de Bayes est un outil fondamental dans la théorie des probabilités, utilisé pour mettre à jour nos croyances sur un événement ***en tenant compte de nouvelles preuves ou informations.***

Définition et formule du théorème de Bayes :

• Le théorème de Bayes décrit la probabilité d'un événement, en se basant ***sur la connaissance préalable des conditions*** qui pourraient être ***liées*** à cet événement.

• La formule du théorème de Bayes est donnée par:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)} \quad \text{avec } P(B) > 0$$

Cette formule exprime la probabilité conditionnelle de A sachant B en termes de la probabilité conditionnelle de B sachant A, multipliée par la probabilité marginale de A, divisée par la probabilité marginale de B.

Application dans les problèmes de classification et de diagnostic:

• Utilisation dans les problèmes de classification, tels que la classification des e-mails en spam ou non spam.

• Application dans les problèmes de diagnostic médical pour estimer la probabilité d'une maladie sachant certains symptômes .

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Théorème de Bayes : Modèles probabilistes

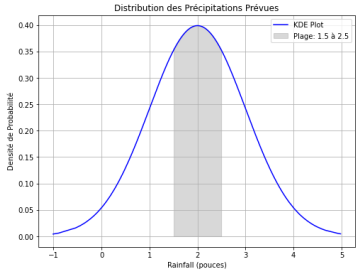
Dans notre étude générale des modèles probabilistes « Bayésiens », nous avons examiné trois variantes principales du classificateur Naive Bayes et leurs domaines d'application spécifiques :



• MultinomialNB : Ce modèle est largement utilisé dans les tâches de classification de texte, telles que la détection de spam, la classification de documents et l'analyse des sentiments. Il est adapté pour traiter des fonctionnalités qui représentent des nombres ou des fréquences, ce qui est commun dans les données textuelles où les mots sont comptés ou leur fréquence est mesurée.



• BernoulliNB : Contrairement à MultinomialNB, BernoulliNB est plus adapté lorsque les caractéristiques sont binaires, c'est-à-dire qu'elles prennent seulement deux valeurs possibles, généralement 0 ou 1. Il est couramment utilisé dans la classification de documents où chaque document est représenté par la présence ou l'absence de certains mots.



• GaussianNB : Ce modèle est utilisé lorsque les caractéristiques sont continuées et suivent une distribution normale, également appelée distribution gaussienne. Il est souvent utilisé dans des domaines tels que la classification biomédicale ou la reconnaissance de caractères, où les caractéristiques sont des mesures continues telles que la taille, le poids, la température, etc.

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

ASSIMILER LES PROBABILITÉS ET DISTRIBUTIONS AVANCÉES

La Détection de Spam avec Naive Bayes :

Contexte :

Nous allons apprendre à construire un modèle de détection de spam en utilisant le classificateur Naive Bayes. Le modèle MultinomialNB il convient au traitement des fonctionnalités qui représentent des nombres ou des fréquences, ce qui est souvent le cas dans les tâches de classification de texte, comme la détection de spam, mais d'autres variantes de modèles Naive Bayes peuvent mieux convenir à différents types de données :

BernoulliNB : adapté lorsque les caractéristiques sont binaires.

GaussianNB : adapté pour les caractéristiques continues qui suivent une distribution normale aussi appelé gaussienne.

Le spam, est un problème courant dans les communications électroniques, et détecter automatiquement ces messages indésirables est une tâche
importante de l'apprentissage automatique.

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

ASSIMILER LES PROBABILITÉS ET DISTRIBUTIONS AVANCÉES

La Détection de Spam avec Naive Bayes :

Appliquez les étapes suivantes pour la construction du système :

- Importation des Bibliothèques :

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import
CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

ASSIMILER LES PROBABILITÉS ET DISTRIBUTIONS AVANCÉES

La Détection de Spam avec Naive Bayes :

1. Chargement et Analyse des Données :

```
# Utilisez pandas pour lire le fichier CSV téléchargé (local):
import pandas as pd
data = pd.read_csv('C:/…/spam.csv')

# Ou sur Google colab charger les données depuis le fichier CSV (Drive)
data = pd.read_csv('/content/drive/MyDrive/ADA_M201_spam/spam.csv')

# Afficher les premières lignes du jeu de données
print(data.head())

# Vérifier les informations sur les colonnes et les valeurs manquantes
print(data.info())
print(data.isna().sum()) #if is Null
```



- Lien de data set Spam/ham format CSV :

https://drive.google.com/file/d/1yEmi4CaCQvHRxOHUmJ13ILf7v4x357lM/view?usp=sharing

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

ASSIMILER LES PROBABILITÉS ET DISTRIBUTIONS AVANCÉES

La Détection de Spam avec Naive Bayes :

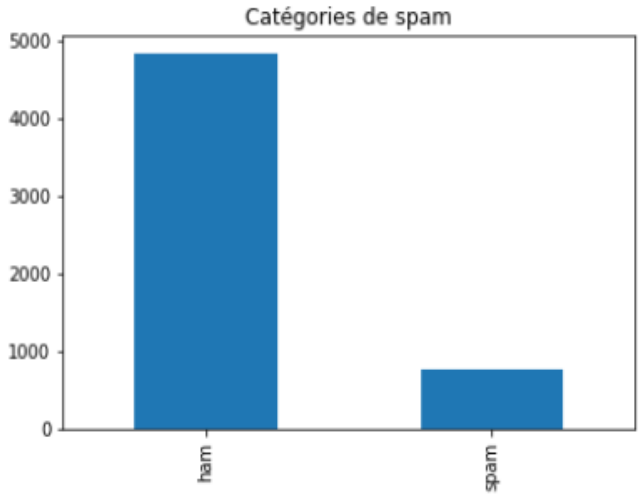
- ***2. Prétraitement des Données :***

```
# Créer une nouvelle colonne 'Spam' avec des valeurs binaires
data['Spam'] = data['Category'].apply(lambda x: 1 if x == 'spam' else 0)

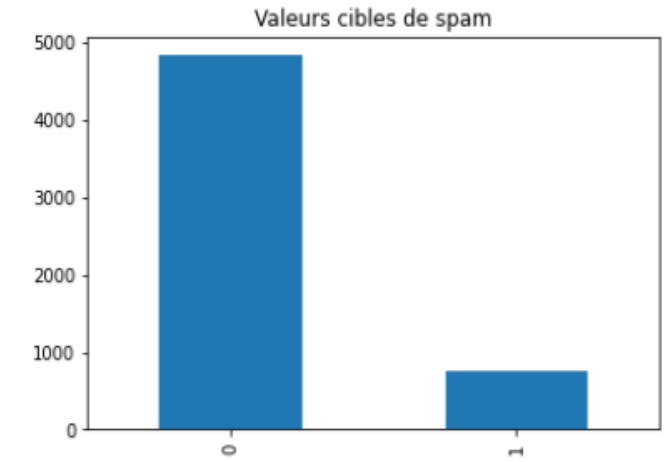
# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(data['Message'], data['Spam'], test_size=0.25)
```

Ce code réalise deux étapes importantes dans la préparation des données :

- Il crée une nouvelle colonne appelée 'Spam' dans le DataFrame 'data' {étant du spam (1) ou non (0)}
- Diviser les données (train, test), {Les données de 'Message' sont utilisées comme caractéristiques (X), et les étiquettes de 'Spam' sont utilisées comme cibles (y)}




```
# Tracer un graphique à barres de la colonne 'Spam'
data['Spam'].value_counts().plot(kind='bar')
plt.title('Target value spam')
plt.show()
```



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

ASSIMILER LES PROBABILITÉS ET DISTRIBUTIONS AVANCÉES

La Détection de Spam avec Naive Bayes :

- ***3. Construction et Entraînement du Modèle :***

```
# Créer un pipeline avec CountVectorizer et MultinomialNB
clf = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('nb', MultinomialNB())
])

# Entraîner le modèle sur les données d'entraînement
clf.fit(X_train, y_train)
```

- ***4. Évaluation du Modèle :***

```
# Évaluer la performance du modèle sur les données de test
accuracy = clf.score(X_test, y_test)
print("Précision du modèle :", accuracy)
```

- ***5. Tests avec de Nouveaux E-mails :***

```
# Définir de nouveaux e-mails à tester
emails = [
    'Sounds great! Are you home now?',
    'Will u meet ur dream soon? Is ur career off 2 a flyng
start? 2 find out free, txt HORO followed by ur star sign,
e. g. HORO ARIES',
    'subscribe now for free',
    'Hi, can you send me the report of last sprint',
    'Testons maintenant cette phrase française',
    'Students of AI will be amazed with this application of
Naive Bayesian, Best Regards M. EL BAKKALI',
]

# Prédire si les e-mails sont du spam ou non
predictions = clf.predict(emails)
print("Prédictions pour les nouveaux e-mails :",
predictions)
```

Expliquer les résultats obtenus !!!!!!!!!!!!

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Imaginons que nous voulions connaître la taille moyenne mondiale, mais mesurer chaque personne est impossible. Nous pouvons donc mesurer un échantillon aléatoire, évitant les biais de sélection.

Plus l'échantillon est grand, plus la moyenne sera proche de la réalité, comme le dit la "loi des grands nombres".

Les simulations Monte Carlo fonctionnent sur le même principe : en prenant des échantillons aléatoires, elles représentent efficacement des situations complexes.



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Pour cette application, nous partirons du principe que nous disposons les tailles de 1000 personnes comme population initiale. Nous utiliserons la méthode de Monte Carlo pour prédire les caractéristiques de cette population en utilisant moins de données, puis nous comparerons les résultats obtenus.

Étapes de l'expérience :

1.Générer les hauteurs de 1000 personnes distribuées normalement autour de 170 cm.

2. Calculer et Afficher les statistiques de la population initiale.

3.Visualiser la distribution de la population initiale.

4.Effectuer une simulation Monte Carlo en échantillonnant aléatoirement 100 personnes (10% de population) à partir de la population initiale.

5.Calculer les statistiques de l'échantillon Monte Carlo.

6.Afficher les statistiques de l'échantillon Monte Carlo.

7.Visualiser la distribution de l'échantillon Monte Carlo.

8.Répéter les calculs, l'affichage et la visualisation pour différentes simulations Monte Carlo en échantillonnant aléatoirement (5% et 0,5%) de la population à partir de la population initiale.

9.Comparaison et synthèse des résultats.

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Générer les hauteurs de 1000 personnes distribuées normalement autour de 170 cm
np.random.seed(42)
population_data = np.random.normal(loc=170, scale=10, size=1000)  # Moyenne de la taille : 170 cm, Écart type : 10 cm

# 2.A Calculer les statistiques de la population initiale
initial_mean = np.mean(population_data)
initial_std = np.std(population_data)

# 2.B Afficher les statistiques de la population initiale et de l'échantillon Monte Carlo
print("Statistiques de la population initiale:")
print("Moyenne :", initial_mean)
print("Écart type :", initial_std)
```

Sortie Attendue :

```
Statistiques de la population initiale:
Moyenne : 170.19332055822326
Écart type : 9.787262077473542
```



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

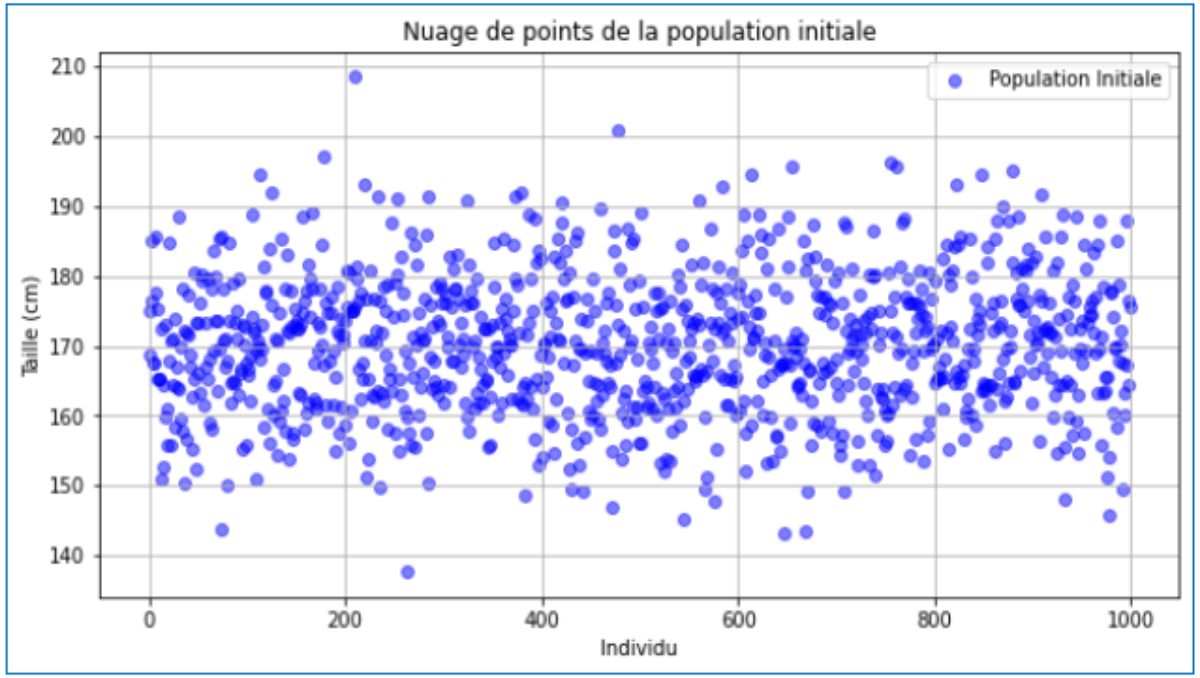
Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

```
# 3. Visualiser la distribution de la population initiale avec un nuage de points
plt.figure(figsize=(10, 5))
plt.scatter(np.arange(len(population_data)), population_data, color='blue', alpha=0.5, label='Population Initiale')
plt.xlabel('Individu')
plt.ylabel('Taille (cm)')
plt.title('Nuage de points de la population initiale')
plt.legend()
plt.grid(True)
plt.show()
```

une explication détaillée sur la création et les types de graphiques est fournie dans la partie suivante

Sortie Attendue :



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Création et Statistiques de la simulation : Ce code effectue une simulation Monte Carlo en sélectionnant aléatoirement 100 personnes à partir de la population initiale. Ensuite, il calcule la moyenne et l'écart type de cet échantillon Monte Carlo et les affiche.

```python
# 4. Effectuer une simulation Monte Carlo en échantillonnant aléatoirement 100 personnes à partir de la population initiale
sample_size = 100
monte_carlo_indices = np.random.choice(len(population_data), size=sample_size, replace=False)
monte_carlo_sample = population_data[monte_carlo_indices]

# 5. Calculer les statistiques de l'échantillon Monte Carlo
monte_carlo_mean = np.mean(monte_carlo_sample)
monte_carlo_std = np.std(monte_carlo_sample)

# 6. Afficher les statistiques de l'échantillon Monte Carlo
print("Statistiques de l'échantillon Monte Carlo:")
print("Moyenne :", monte_carlo_mean)
print("Écart type :", monte_carlo_std)
```

Sortie Attendue :

```
Statistiques de l'échantillon Monte Carlo:
Moyenne : 171.38470622599397
Écart type : 10.384381004252985
```

100 comme taille d'échantillon représente 10 % de la population

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

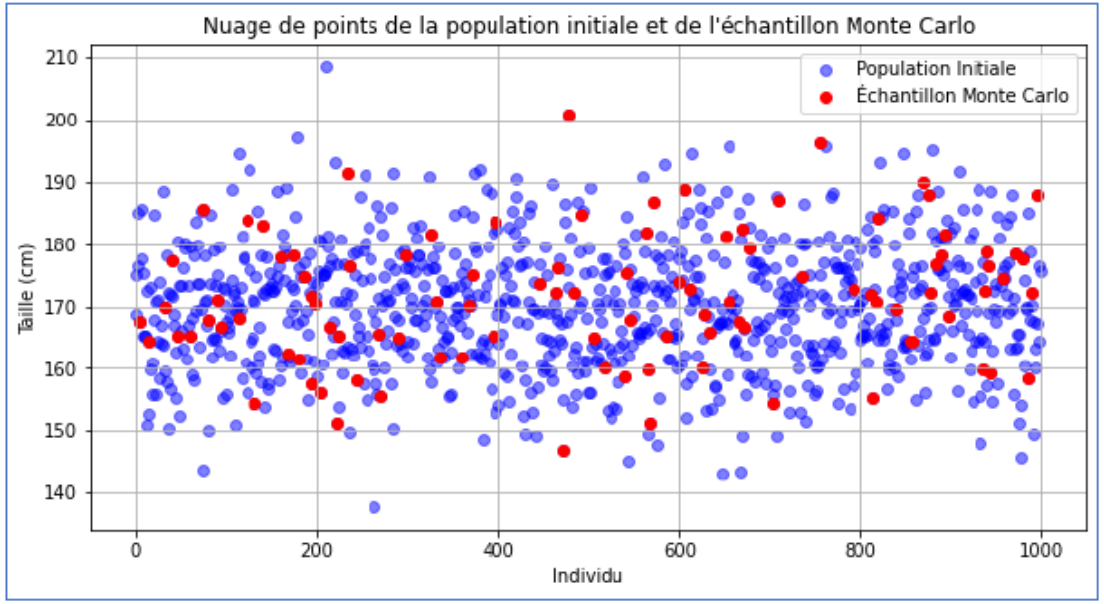
Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Affichage :

```
# 7. Visualiser la distribution de la population
initiale et de l'échantillon Monte Carlo ensemble
avec un nuage de points
plt.figure(figsize=(10, 5))
plt.scatter(np.arange(len(population_data)),
population_data, color='blue', alpha=0.5,
label='Population Initiale')
plt.scatter(monte_carlo_indices, monte_carlo_sample,
color='red', label='Échantillon Monte Carlo')
plt.xlabel('Individu')
plt.ylabel('Taille (cm)')
plt.title('Nuage de points de la population initiale
et de l\'échantillon Monte Carlo')
plt.legend()
plt.grid(True)
plt.show()
```

Sortie Attendue :



10 % de la population

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

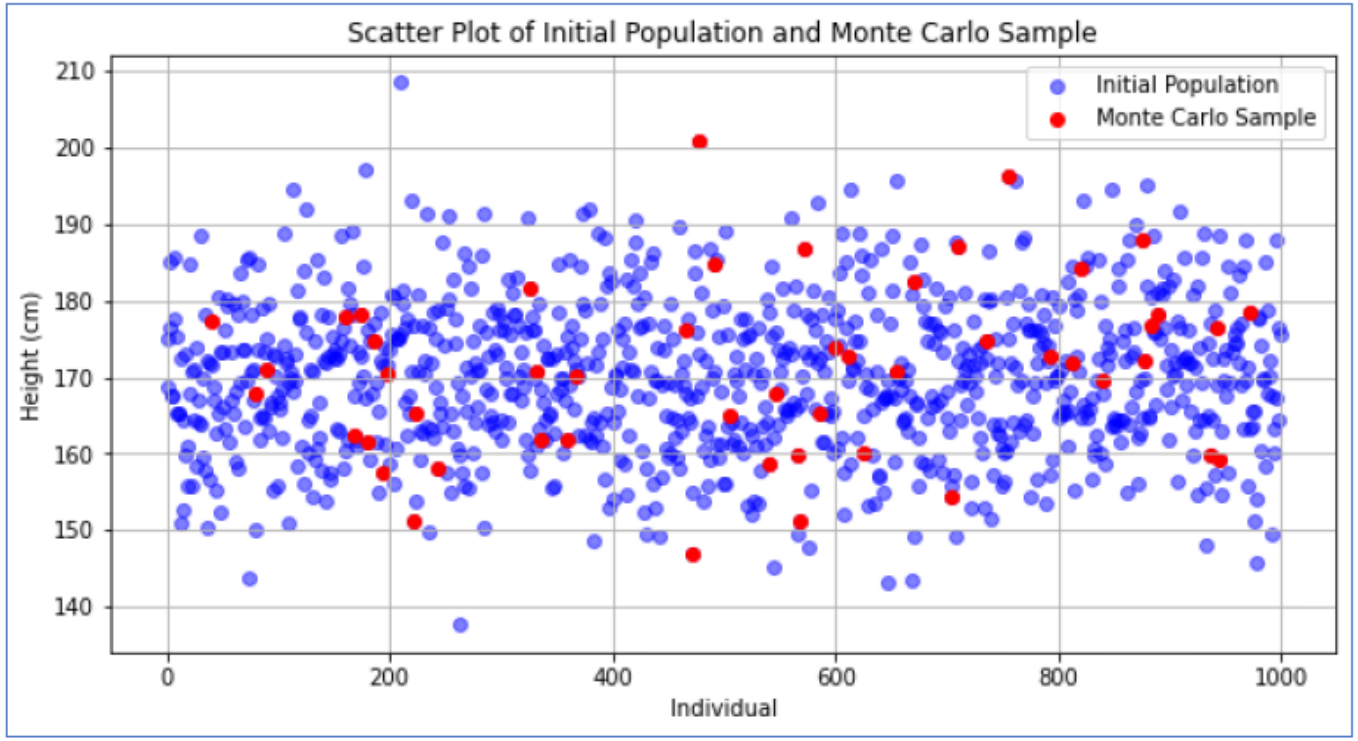
Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Affichage pour 5% :

Remarquez comment même en sélectionnant seulement 5 % des données, nous avons obtenu un échantillon représentatif. Pour obtenir ces résultats, nous avons simplement changé la taille de l'échantillon à 50, ce qui correspond à 5 % des données.

Résultats Attendue :




```
Monte Carlo 5% Moyenne : 170.86293396896747
```

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

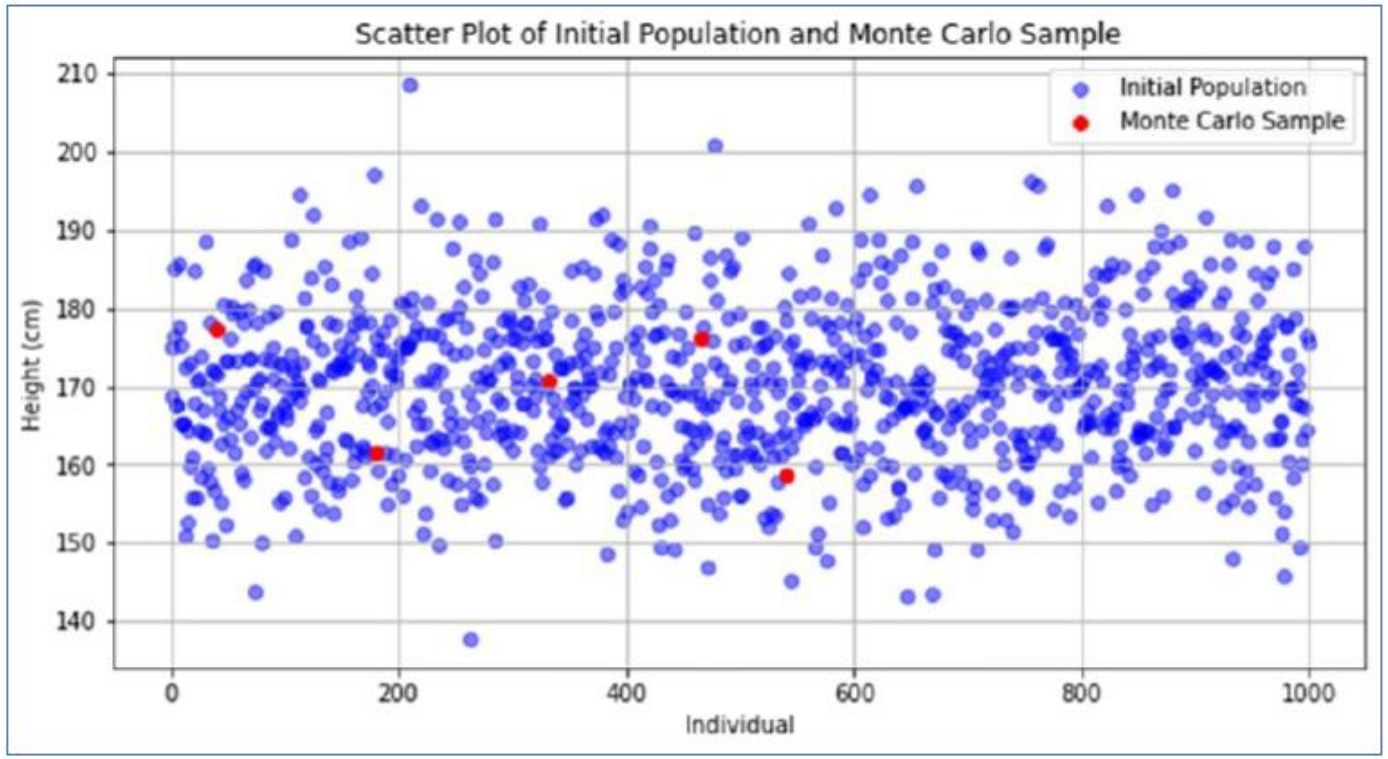
Application pratique avec Python : Simulation Monte Carlo

Affichage pour 0,5% :

Nous avons remarqué qu'avec seulement 0,5 % des données, nous avons également réussi à obtenir des données représentatives, mais avec une précision moindre dans les mesures statistiques.

Pour obtenir ces résultats, nous avons simplement changé la taille de l'échantillon à 5, ce qui correspond à 0,5 % des données.

Résultats Attendue :




```
Monte Carlo 0.5% Moyenne : 168.88314685431732
```

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Notions avancées de probabilité

Application pratique avec Python : Simulation Monte Carlo

Synthèse des résultats :

• L'expérience a démontré l'efficacité de la méthode Monte Carlo dans la prédiction des caractéristiques de la population avec différents échantillons de données.

• La comparaison des résultats montre que même avec des échantillons de seulement 5 % et 0,5 % des données, nous obtenons des données représentatives.

• La précision des mesures statistiques diminue avec la diminution de la taille de l'échantillon, soulignant l'importance d'une taille d'échantillon plus grande pour des estimations plus précises.

• La méthode Monte Carlo fonctionne particulièrement bien lorsque les données suivent une distribution normale et sont centrées, ce qui améliore sa précision.

• La plupart des données dans de nombreux domaines suivent cette loi de distribution normale, renforçant l'applicabilité et l'efficacité de la méthode de Monte Carlo dans divers scénarios d'analyse de données.